X-Raying AGN - Lab Notes

Bradley Day

2025-05-19

Table of contents

Preface 4

I Teaching Block 1 5

1 Week 1 6
- 1.1 2026/09/21 . . . 6
- 1.2 2026/09/22 . . . 6
  - 1.2.1 Meeting Minutes . . . 6
- 1.3 2026/09/23 . . . 7
  - 1.3.1 Reading - K. Nandra (2007) [1] . . . 7
- 1.4 2026/09/26 . . . 8
  - 1.4.1 Reading - Fabian et al. (2002) [2] . . . 8

2 Week 2 10
- 2.1 2026/09/29 . . . 10
  - 2.1.1 First Investigations Into Data Reduction . . . 10
  - 2.1.2 Meeting with Supervisor . . . 10
  - 2.1.3 Meeting Minutes . . . 11
  - 2.1.4 Misc Notes . . . 11
- 2.2 2026/09/30 . . . 11
  - 2.2.1 Intro to Linux Meeting (w/Rhys Morris) . . . 11
- 2.3 2026/10/01 . . . 11
  - 2.3.1 XMM-SAS and Julia Setup . . . 11
- 2.4 2026/10/02 . . . 12
  - 2.4.1 XMM-Newton Data Reduction . . . 12
- 2.5 2026/10/03 . . . 15
- 2.6 2026/10/04 . . . 17

3 Week 3 23
- 3.1 2026/10/05 . . . 23
  - 3.1.1 Comparing the bash script to Julia . . . 23
- 3.2 2026/10/07 . . . 24
  - 3.2.1 Meeting with Supervisor . . . 24
- 3.3 2026/10/09 . . . 25
  - 3.3.1 Reprocessing the 0693781301 Dataset Using the PN Instrument . . . 25
- 3.4 2026/10/10 . . . 29
  - 3.4.1 A Julia Script for Spectrum Plotting . . . 29
- 3.5 Summary of Data Reduction Commands . . . 32
  - 3.5.1 Reprocessing . . . 32
  - 3.5.2 GTI Filtering . . . 32
  - 3.5.3 Region Selection . . . 32
  - 3.5.4 Spectrum Extraction . . . 33
  - 3.5.5 Pileup Check . . . 33
  - 3.5.6 Response Files and Cleanup . . . 34

3.5.7 Test Data 34

4 Week 4 **35**

4.1 Initial Fitting Attempt Using XSPEC 35

References **37**

Preface

This book details my research during the final year of my degree at the University of Bristol. Thank you to Professor Andrew Young for supervising and my project partner Jacob Taylor for his work.

This is a Quarto book.

To learn more about Quarto books visit https://quarto.org/docs/books.

Part I

Teaching Block 1

1 Week 1

1.1 2026/09/21

Lab book set up and initialised as a Github page. Meeting with supervisor arranged for 2026/09/22.

1.2 2026/09/22

Plans/questions for meeting:

- Confirm that Quarto is allowed for writing lab book
  - Yes
- Get connected to `solva`
  - This will need to be after Rhys has set things up
  - Note 2026-09-23 have connected to solva, however will wait until Rhys has given an induction to use
- Make sure GitHub set up properly so we can commit/push to it
- Confirm scope of project and form initial plan with Jacob
  - Fitting models?
    - XSPEC?
    - Julia (SpectralFitting.jl)?
    - Other?
  - Data Reduction
    - Using XMM-Newton data so will need to become familiar with XMM SAS
    - Are we using this on solva?
    - Note to self: see project plan on Github for more resources

1.2.1 Meeting Minutes

- XMM is not a survey telescope so even random selections will usually be interesting objects
- Andy will add a link to Nandra's paper on the Github repo for bright sources
  - Read this paper
- Browse paper(s) to find interesting sources
  - NGC4151?
  - Make sure to note when and how long observations were taken
  - HEASARC archive should let us search for particular objects
  - Download data and go through the ABC guide
- Before next meeting:
  - Look into candiadate AGN

    - Look at Tartarus database?
  - Read papers provided by Andy
  - Look through XMM SAS ABC guide

1.3 2026/09/23

1.3.1 Reading - K. Nandra (2007) [1]

- Covers 37 observations of 26 Seyfert galaxies from XMM-Newton
  - All show Iron K emission, which makes this paper useful for candidate selection for our research
  - Around 45% of the data was best fit with a relativistically blurred reflection model
    - Some objects may not show the broadening?
- General background information
  - Iron line emission broadening predicted in Fabian et al (1989)
  - This was alongside the discovery that the emission likely comes from the accretion disk
  - Useful reads?
    - Fabian (1989)
    - George, Nandra & Fabian (1990)
    - Tanaka et al. (1995)
- Data Reduction
  - They give a lot of detail on their data reduction which may be useful further down the line
  - Source taken from a circle with radius 700 detector pixels
  - There are several windowing modes on the CCD which will be important to note
- Model
  - They found that the minimum set of models was:
    - Power law continuum (modelling the corona)
    - Reflection model (e.g. XILLVER)
    - Can use relativistic blurring on this however not necessarily needed
    - Ionized absorber model
  - The models used are covered in depth, including how they selected if and how to apply the absorber models
    - This will be useful later down the line to give some direction when fitting
  - They modelled the emission using a broad and narrow component, this may be useful to consider?
- Potential candiadate list (from Figure 3 of Nandra et al.)
  - NGC3516(2)
    - Very broad emission centred around the Fe Kα line
  - NGC3783(2)
    - Quite a clean spectrum with low noise/small errors
  - NGC4151(2) or NGC4151(3)
    - Clean spectrum with broad emission

1.4 2026/09/26

1.4.1 Reading - Fabian et al. (2002) [2]

1.4.1.1 Background

- Presents preliminary results from a 325 ks observation of MCG-6-30-15
  - Data taken from both the EPIC MOS cameras and the EPIC pn camera in small window mode with a medium filter
- Very clear detection of a strong, broad, and skewed iron line
- Spectrum characterised well by a steep emissivity profile within $6r_g$ and a flatter profile beyond
- Inner radius of emission appears to lie around $2r_g$
  - This was measured from the fit, where the inner radius of the disc was a fitting parameter
  - The measurements are dependent on an assumed power-law continuum and lack of complex absorption above 2.5 keV
- ASCA observation showed emission from an accretion disc extending from $6 - 40r_g$ with an inclination of around 30°
  - Measured from Fe-kα line
- The red wing of the line extends below 4 keV which can be explained by the disc extending within 6 r_g
  - Why is this??
  - This implies rapidly spinning black hole (as ISCO shrinks with a_*)

1.4.1.2 Fitting

- Several models were used to fit the data, including a phenomenological model (parametersing the iron line with power law, Compton reflection, a narrow Gaussian at 6.4 keV and a broad Gaussian which was free)
  - This provided a good overall fit with $\chi^2 = 984.1$ for 1034 degrees of freedom
    - How are the number of dof determined?
  - This was improved by allowing the narrow Gaussian to have some width, giving an "excellent" fit
- Second model used the Laor (1991) emission line profile
  - Fits were insensitive to the outer radius of the disc, which was thus fixed at $400r_g$
  - Free parameters were R_{in}, the inclination, and the emissivity as a function of radius (parametrised as R^{-q})
  - Almost comparable χ^2 to the first model with two broad Gaussians
    - This falls short as with a simple power law emissivity, the red wing and narrow core of the line cannot be modelled simultaneously
- Further improvements were made including
  - Addition of an iron K-edge
    - The K-edge is a sudden increase in x-ray absorption [FIND CITATION FOR THIS, wiki for now]
    - This significantly improved the fit
    - The K-edge however must be explained physically, several attempts were made
      1. Neutral iron along the line of sight
         - This would require strong iron-L absorption, which is not present

2. Neutral iron outside the line of sight
   · Reflection off of distant material which would require strong reflection
   · This is inconsistent with the weak, narrow 6.4 keV iron line
3. Thick material partially covering the line of sight
   · This gave a comparable fit to the edge
4. Sako et al. (2002) suggested significant absorption due to inner-shell transitions of ionised iron
   · Explored using an absorption line model, giving a good fit
5. The complexity around 7.0 keV is emission
   · Recombination emission by H-like iron

∗ It was not possible to determine whether the complexity was due to absorption or emission and both were thus considered

– Use of a broken power law
  ∗ This is not physically self-consistent
    · Ignores Fe Kβ at 7.05 keV
    · The high EW is not consistent with $R_{\rm refl} = 1,\ i = 30^{\circ}$, solar abundances and neutral
    · High abundance was investigated using 3× the solar abundances which improved the fit significantly

- Spectral variability was investigated by taking the ratio of the spectra resulting from two 10 ks intervals from the maximum and minimum flux periods
  – Taking the ratio of the two spectra showed clear signs
  – The iron line core is stronger in the low-flux spectrum
- The models assume that the effects of ionized absorption are negligible above 2.5 keV, correct for absorption rising from low-Z elements, but may not be true if e.g. Si, S are present

2 Week 2

2.1 2026/09/29

2.1.1 First Investigations Into Data Reduction

- Can now use solva, and XMM-SAS is installed.
- Have got a script to automate the reduction thanks to Darius Michienzi
  - GitHub
  - This script is written in Julia so requires a Julia installation on solva
  - Currently running the command `julia` yields the error `-bash: julia: command not found`
  - Need to check with Rhys Morris whether Julia is installed, if so how can we access it, and if not whether we can install it
  - For now will proceed with manual reduction to learn the process, as the script may not cover all of our needs, so worth learning how it works under the hood
- Using the XMM-SAS ABC guide
  - Using dataset with OBSID 0761670101
  - Following instructions for EPIC (IMAGING Mode)
  - The first set of commands are

```
cd ODF
setenv SAS_ODF /full/path/to/ODF/directory/
setenv SAS_ODFPATH /full/path/to/ODF/directory/
cifbuild
```

  - This does not currently work, the `setenv` command seems to be for a `csh` shell, while solva uses `bash`
    - This can be worked around by using e.g. `export SAS_ODF="/full/path/to/ODF/directory/"`
  - The `cifbuild` command also does not currently work, with the terminal giving the error that the command was not found

2.1.2 Meeting with Supervisor

- Questions
  - In Fabian et al. (2002) [2] it says that the red wing of the line extending below 4 keV can be explained by the disc extending within 6 r_g
    - Why is this?
    - Is this just what their fit says?
  - Maybe start considering how we are going to fit the data?
    - It may be possible to get a headstart setting up pipelines for it
    - Discuss with Supervisor/Jacob if its worth using Julia (`SpectralFitting.jl`) or `xspec`?
      - Or something else?
    - If using Julia, would it be worth designing our model?
      - Is there anything we can gain from this? More freedom in the parameters etc?

2.1.3 Meeting Minutes

- When fitting, will use XSPEC
  - Can create a table model if we need to use more abstract metrics etc.
  - Group working on catalogue of metrics have managed to get Gradus working for naked singularities (superspinning black holes)
    - May be able to use this to fit non-standard descriptions to the data, and collaboratively investigate the cosmic censorship conjecture
- Downloading data from XMM-Newton Archive
  - Through legacy interface, create download script (gives `wget` script)
  - May be able to enter RA/DEC to find OBSIDs
- Meeting was successful with Andy's other group
  - They are working on cataloguing spacetimes and investigating the cosmic censorshsolvaip conjecture
    - This is the conjecture that naked singularities do not exist
    - They may be able to compute iron line profiles with this, allowing us to fit this supposedly unphysical model to observations
    - If doing this we will need to construct a table model for XSPEC
  - Groupchat set up with Fionn & Max which should allow us to compare notes and collaborate on issues

2.1.4 Misc Notes

- Recurring meeting set for 11:00 on Fridays with Jacob to keep each other in the loop and discuss issues

2.2 2026/09/30

2.2.1 Intro to Linux Meeting (w/Rhys Morris)

- star.bris.ac.uk/project_access

2.3 2026/10/01

2.3.1 XMM-SAS and Julia Setup

Once logged into solva, XMM-SAS and Julia can be set up by running:

```
module load julia/1.9.2
```

for Julia, which can be tested by simply running `julia`. XMM-SAS requires a few more lines of setup:

```
# First initialise HEASOFT
export HEADAS=/opt/local/rocky8/heasoft-6.36/x86_64-pc-linux-gnu-libc2.28/
alias heainit=". $HEADAS/headas-init.sh"
heainit

# Then initialise XMM-SAS
module load anaconda/3-2024
export SAS_DIR=/opt/local/rocky8/xmmsas_20230412_1735
source $SAS_DIR/sas-setup.sh
```

Running `sasversion` should output

```
sasversion:- Executing (routine): sasversion  -w 1 -V 4
sasversion:- sasversion (sasversion-1.3)  [xmmsas_20230412_1735-21.0.0] started:  2026-10-
01T10:13:30.000
sasversion:- XMM-Newton SAS release and build information:

SAS release: xmmsas_20230412_1735-21.0.0
Compiled on: Sun Apr 16 21:03:42 CEST 2023
Compiled by: sasbuild@sasbld05.iuser.lan
Platform   : RHEL8.6

SAS-related environment variables that are set:

SAS_DIR = /opt/local/rocky8/xmmsas_20230412_1735
SAS_PATH = /opt/local/rocky8/xmmsas_20230412_1735

sasversion:- sasversion (sasversion-1.3)  [xmmsas_20230412_1735-21.0.0] ended:    2026-10-
01T10:13:30.000
```

2.4 2026/10/02

2.4.1 XMM-Newton Data Reduction

In order to avoid running all of the above commands upon every login session, the following was completed:

With Julia and anaconda loaded, the `module save` command was run, which saves the loaded module configuration to a file. Then, editing `.bashrc` via `nano .bashrc`, the following lines were added to the file:

```
#========ADDED BY USER========#

# Load Modules
module restore

# Environment variables
export HEADAS=/opt/local/rocky8/heasoft-6.36/x86_64-pc-linux-gnu-libc2.28/
export SAS_DIR=/opt/local/rocky8/xmmsas_20230412_1735

# Init HEASOFT
alias heainit=". $HEADAS/headas-init.sh"
```

```
heainit

# Init SAS
alias sasinit="source $SAS_DIR/sas-setup.sh"

#========ADDED BY USER========#
```

This initialises all of the environment variables upon login, and all that is needed to do is to run `sasinit`.

Data was downloaded using a `wget` command as follows:

```
wget -v -q -nH --no-check-certificate --cut-dirs=4 -r -l0 -w1 -c -N -np -R 'index*'
 ↪  -erobots=off https://heasarc.gsfc.nasa.gov/FTP/xmm/data/rev0//0123700101/ODF/
```

It is currently believed that this can be reused

2.4.1.1 Pipeline Reprocessing

With this working, I began working through the XMM-SAS ABC guide for data reduction. Everything up to the end of the making a Light Curve section of the ABC guide is automated by the script provided by Darius Michienzi.

Run the script using

```
# Unzip any zipped files
gzip -d *.gz

# Run the script
julia /home/jq23394/XMM-data-reduction.jl
```

in the ODF directory. Set useful environment variables using:

```
export SAS_ODFPATH="$(pwd)"
export SAS_CCFPATH="/opt/local/XMM/ccf/"
export SAS_CCF="$SAS_ODFPATH/ccf.cif"
export SAS_ODF="$SAS_ODFPATH/$(ls *SUM.SAS)"
```

evselect table=mos1_filt.fits withrateset=Y rateset=mos1_ltcrv.fits maketimecolumn=Y timebinsize=100 makeratecolumn=yes

2.4.1.2 Time Filtering

The next step is to apply time filters, which can be done by looking at either the rate or the time intervals. To filter by rate, use the syntax

```
tabgtigen table=mos1_ltcrv.fits gtiset=gtiset.fits expression='RATE<=6'
```

where the expression `RATE<=6` selects all regions with a rate below 6 ct/s. To filter by time, use the syntax

```
tabgtigen table=mos1_ltcrv.fits gtiset=gtiset.fits timecolumn=TIME expression='TIME <=
 ↪  73227600'
```

where the expression `TIME<=73227600` selects the region below a given timestamp. Conditions can be combined using e.g. `(EXP1)&&(EXP2)&&!(EXP3)` which means that regions will be taken from expressions 1 and 2, and ignore anything that matches expresssion 3.

Either of these methods will generate a gti file, which can be used as

```
evselect table=mos1_filt.fits filtertype=expression filteredset=mos1_filt_time.fits
 ↪  expression='GTI(gtiset.fits,TIME)'
```

2.4.1.3 Image and Source Detection

After this, run

```
atthkgen atthkset=attitude.fits
```

Next, we produce images using binsizes of 22 or 82 for the MOS or PN cameras respectively. This corresponds to the size of the pixels of each on the sky, being 1.1" and 4.1", respectively.

```
evselect table=mos1_filt_time.fits withimageset=yes imageset=mos1-s.fits
 ↪  imagebinning=binSize xcolumn=X ximagebinsize=22 ycolumn=Y yimagebinsize=22
 ↪  filtertype=expression expression='(FLAG == 0)&&(PI in [300:2000])'

evselect table=mos1_filt_time.fits withimageset=yes imageset=mos1-h.fits
 ↪  imagebinning=binSize xcolumn=X ximagebinsize=22 ycolumn=Y yimagebinsize=22
 ↪  filtertype=expression expression='(FLAG == 0)&&(PI in [2000:10000])'

evselect table=mos1_filt_time.fits withimageset=yes imageset=mos1-all.fits
 ↪  imagebinning=binSize xcolumn=X ximagebinsize=22 ycolumn=Y yimagebinsize=22
 ↪  filtertype=expression expression='(FLAG == 0)&&(PI in [300:10000])'
```

replacing the options `mos1_filt_time.fits` and the image bin sizes as necessary.

Initial runs of this resulted in the error

```
** evselect: error (ExprColGenError), Error: UnknownColumn: No column named `X' in table
 ↪  `/tmp/fileYpufn0:EVENTS'
Column `X' does not exist in table `/tmp/fileYpufn0:EVENTS'
```

Using `ftlist` on the filtered events file showed that the columns `X` and `Y` did not exist. Due to this the data used in the ABC guide was downloaded to test that the instructions work in the first place. It was noticed that during the reprocessing steps `emproc` and `epproc`, the messages

```
Column X does not exist - creating
Column X does not exist - creating
```

would frequently appear. The issue may thus be an error during this step. Following through the instructions for the example dataset yielded successful results, thus more investigation is required.

2.5 2026/10/03

With the confirmation that the instructions work for the example dataset (OBSID=0123700101), I returned to a MCG-6-30-15 dataset (OBSID=0693781301) and manually ran each command rather than using the script. I also attempted the process with the observation 0111570101, however this data seemed much less promising. The process was as follows:

```
# Enter the ODF directory
cd ODF

# Unzip
gzip -d *.gz

# Set environment variables and build cif file
export SAS_ODFPATH="$(pwd)"
export SAS_ODF="$(pwd)"
export SAS_CCFPATH="/opt/local/XMM/ccf/"
cifbuild

# Storing the ccf file and running odfingest
export SAS_CCF="$SAS_ODFPATH/ccf.cif"
odfingest

# Storing the odf file as an environment variable
export SAS_ODF="$SAS_ODFPATH/$(ls *SUM.SAS)"

# Make and enter processing directory
mkdir ../PROC
cd ../PROC

# Reprocessing pipelines
emproc
epproc

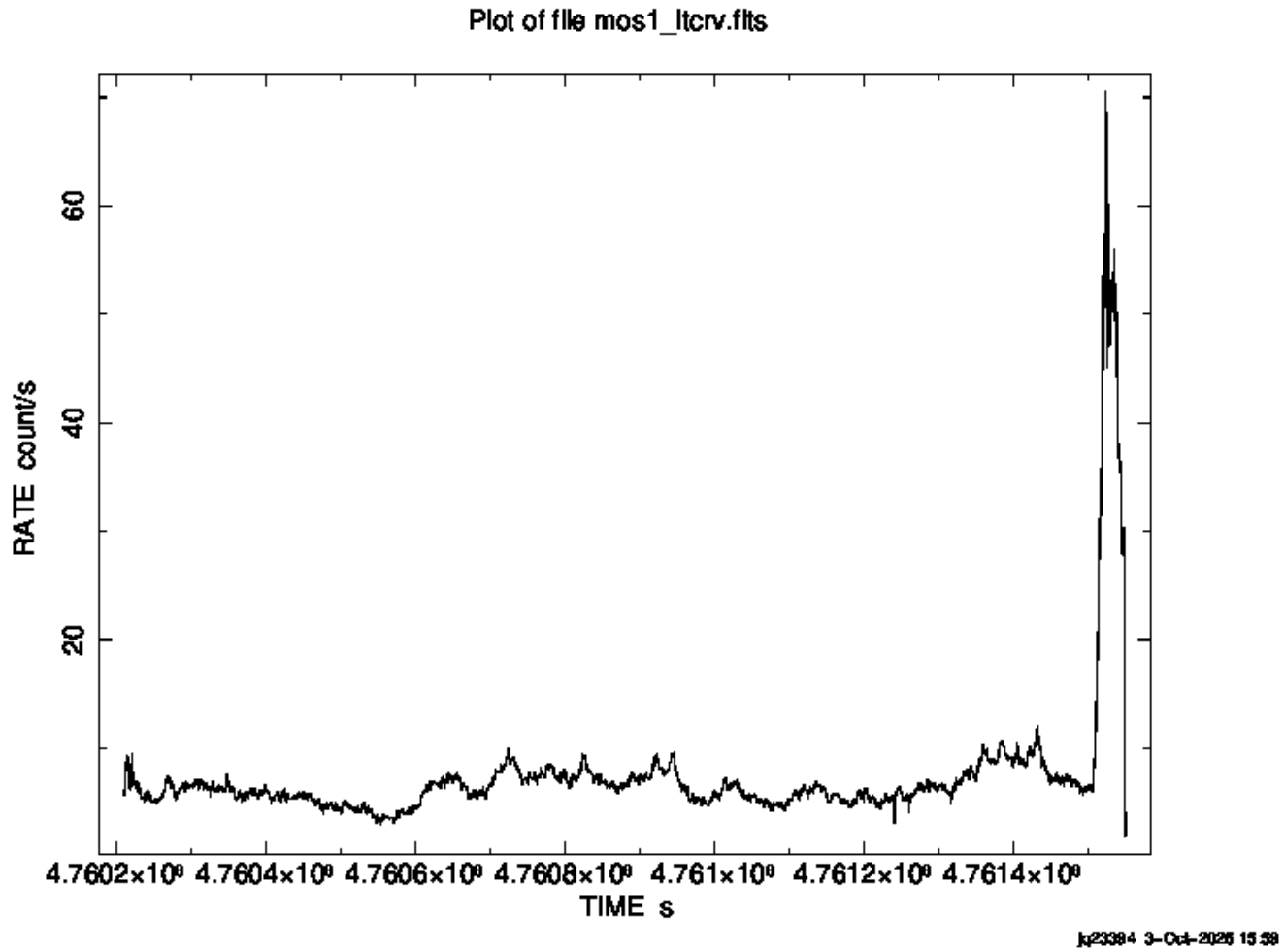
# Make a symbolic link for easy command use
ln -s *_EMOS1_*ImagingEvts.ds mos1.fits

# Apply standard filters
evselect table=mos1.fits filtertype=expression filteredset=mos1_filt.fits
 ↪  expression='(PATTERN <= 12) && (PI in [200:12000]) && #XMMEA_EM'

# Make a light curve
evselect table=mos1_filt.fits withrateset=Y rateset=mos1_ltcrv.fits maketimecolumn=Y
 ↪  timebinsize=100 makeratecolumn=yes

# Plot the light curve
fplot mos1_ltcrv.fits TIME RATE - /PS EXIT
```

The lightcurve is shown below



There was a big flare at the end of the observation, which was filtered out with the condition that RATE must be ≤ 15.

```
# Creating the GTI (Good Time Interval) file for time filtering
tabgtigen table=mos1_ltcrv.fits gtiset=gtiset.fits expression='RATE<=15'

# Filtering the events file using the GTI
evselect table=mos1_filt.fits filtertype=expression filteredset=mos1_filt_time.fits
 ↪  expression='GTI(gtiset.fits,TIME)'

# Creating an attitude file
atthkgen atthkset=attitude.fits

# Creating an image for soft X-ray detections
evselect table=mos1_filt_time.fits withimageset=yes imageset=mos1-s.fits
 ↪  imagebinning=binSize xcolumn=X ximagebinsize=22 ycolumn=Y yimagebinsize=22
 ↪  filtertype=expression expression='(FLAG == 0)&&(PI in [300:2000])'

# Creating an image for hard X-ray detections
evselect table=mos1_filt_time.fits withimageset=yes imageset=mos1-h.fits
 ↪  imagebinning=binSize xcolumn=X ximagebinsize=22 ycolumn=Y yimagebinsize=22
 ↪  filtertype=expression expression='(FLAG == 0)&&(PI in [2000:10000])'

# Creating an image for all X-ray detections
```

```
evselect table=mos1_filt_time.fits withimageset=yes imageset=mos1-all.fits
 ↪  imagebinning=binSize xcolumn=X ximagebinsize=22 ycolumn=Y yimagebinsize=22
 ↪  filtertype=expression expression='(FLAG == 0)&&(PI in [300:10000])'

# Running the source detection algorithm on the data
edetect_chain imagesets='mos1-s.fits mos1-h.fits' eventsets='mos1_filt_time.fits'
 ↪  attitudeset=attitude.fits pimin='300 2000' pimax='2000 10000' ecf='0.878 0.220'
```

The process is currently getting stuck on the `edetect_chain` step.

2.6 2026/10/04

Due to the process getting stuck on the source detection `edetect_chain` step, it was decided to skip this step and manually select the source region, given it was easy to find in this data. The selection was done in ds9 via

```
ds9 mos1_filt_time.fits &
```

The image, alongside the source and background regions, were selected as shown below

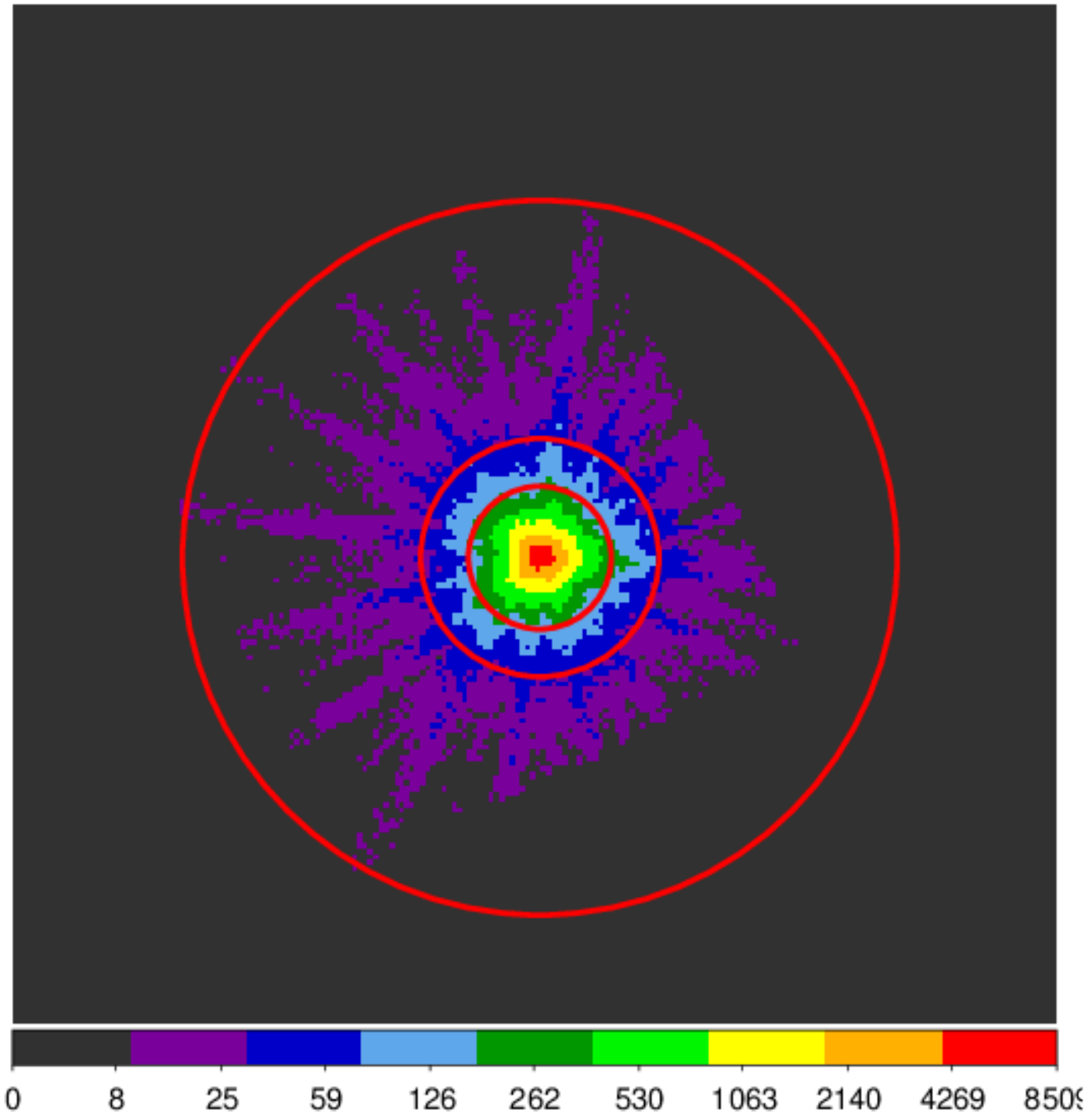


Figure 2.1: The regions used for spectrum extraction. The outer two circles were used to select an annulus for the background. The inner circle was used for the source region. The regions were centered at (25631.2,23903,300) with radii of 300, 500, and 1500 pixels.

An exposure map was made from the hard X-ray image via

```
# Generate Exposure Map
eexpmap imageset=mos1-h.fits attitudeset=attitude.fits eventset=mos1_filt_time.fits
 ↪  expimageset=mos1_expmap_2-10keV.fits pimin=2000 pimax=10000
```

The spectra could then be extracted as

```
# Extract the Source Spectrum
evselect table='mos1_filt_time.fits' energycolumn='PI' filteredset='mos1_filtered.fits'
 ↪  filtertype='expression' expression='((X,Y) in CIRCLE(25631.2,23903,300))'
 ↪  spectrumset='mos1_pi.fits' spectralbinsize=5 withspecranges=yes specchannelmin=0
 ↪  specchannelmax=11999

# Extract the Background Spectrum
evselect table=mos1_filt_time.fits energycolumn='PI' filteredset='bkg_filtered.fits'
 ↪  filtertype='expression' spectrumset='bkg_pi.fits' spectralbinsize=5 expression='((X,Y)
 ↪  in CIRCLE(25631.2,23903,1500))&&!((X,Y) in CIRCLE(25631.2,23903,500))'
 ↪  withspecranges=yes specchannelmin=0 specchannelmax=11999
```

The spectrum was checked for pileup with

```
# Check for Pileup
epatplot set=mos1_filtered.fits plotfile=mos1_epat.ps useplotfile=yes withbackgroundset=yes
 ↪  backgroundset=bkg_filtered.fits
```

which output the following graph

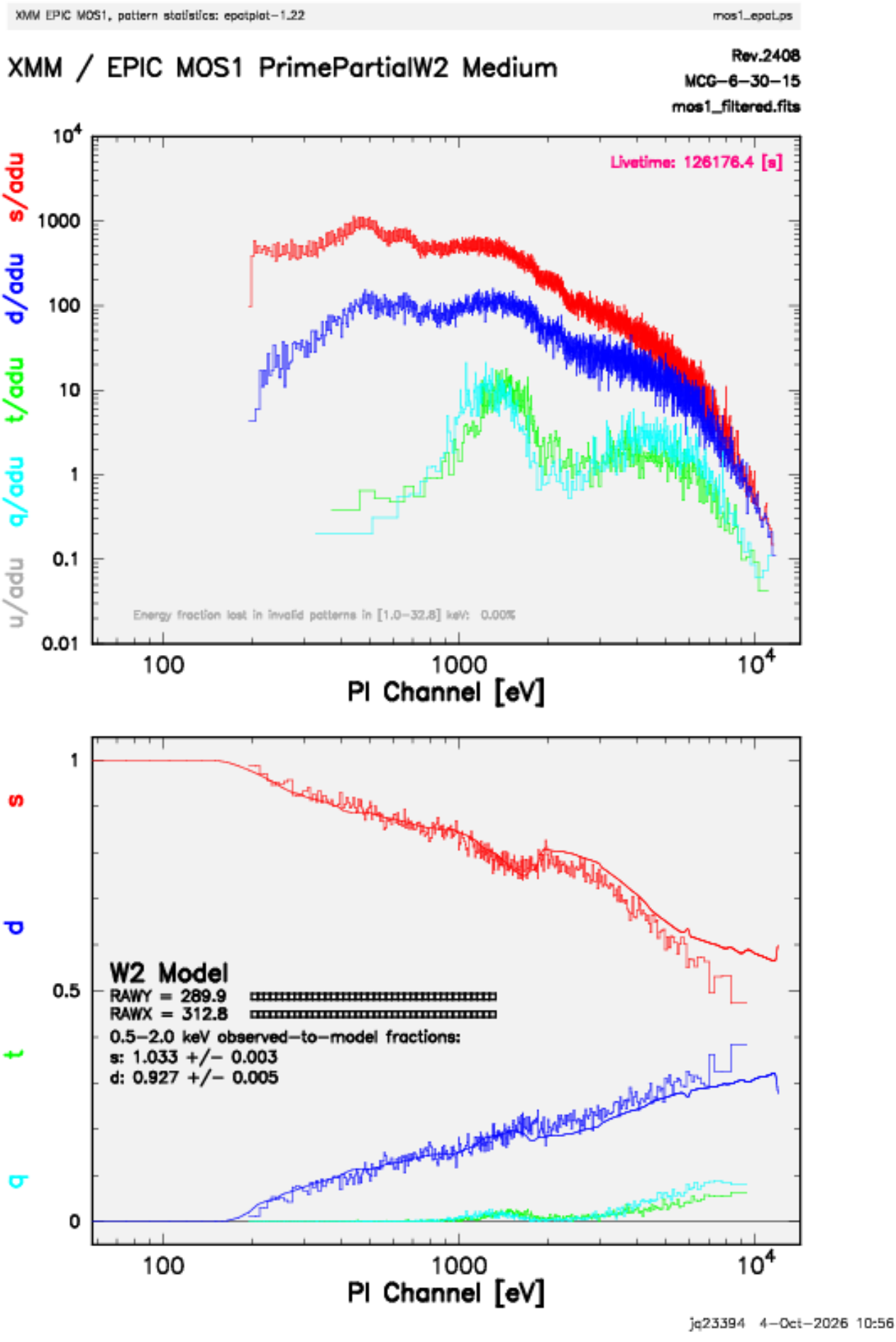


Figure 2.2: The output of `epatplot`. The modelled distributions agree well with the observed distributions (as seen in the lower panel) and therefore pileup can be neglected here.

```
# Find the source and background region areas for the ancillary file
backscale spectrumset=mos1_pi.fits badpixlocation=mos1_filt_time.fits
backscale spectrumset=bkg_pi.fits badpixlocation=mos1_filt_time.fits

# Generate the Photon Redistribution Matrix
rmfgen rmfset=mos1_rmf.fits spectrumset=mos1_pi.fits

# Generate the Ancillary File
arfgen arfset=mos1_arf.fits spectrumset=mos1_pi.fits withrmfset=yes rmfset=mos1_rmf.fits
 ↪  withbadpixcorr=yes badpixlocation=mos1_filt_time.fits
```

With these files successfully generated, it was possible to plot the spectrum through a quick Julia script, making use of the `SpectralFitting.jl` package

```
using SpectralFitting, Plots

function LoadData(
    spectrum::String, background::String, response::String, ancillary::String;
    dataRange::Tuple{Real, Real}=(3,10)
    )

    data = OGIPDataset(
        spectrum;
        background = background,
        response = response,
        ancillary = ancillary
    )

    # Regrouping, normalising, dropping bad channels and curtailing
    regroup!(data)
    normalize!(data)
    drop_bad_channels!(data)
    mask_energies!(data, dataRange...)
end

# Setting the instrument as a variable for generaliseability
instrument = "mos1"

data = LoadData(
    "$(instrument)_pi.fits",
    "bkg_pi.fits",
    "$(instrument)_rmf.fits",
    "$(instrument)_arf.fits"
)

plot(data)
```

Using the data reduced here, the spectrum below was produced

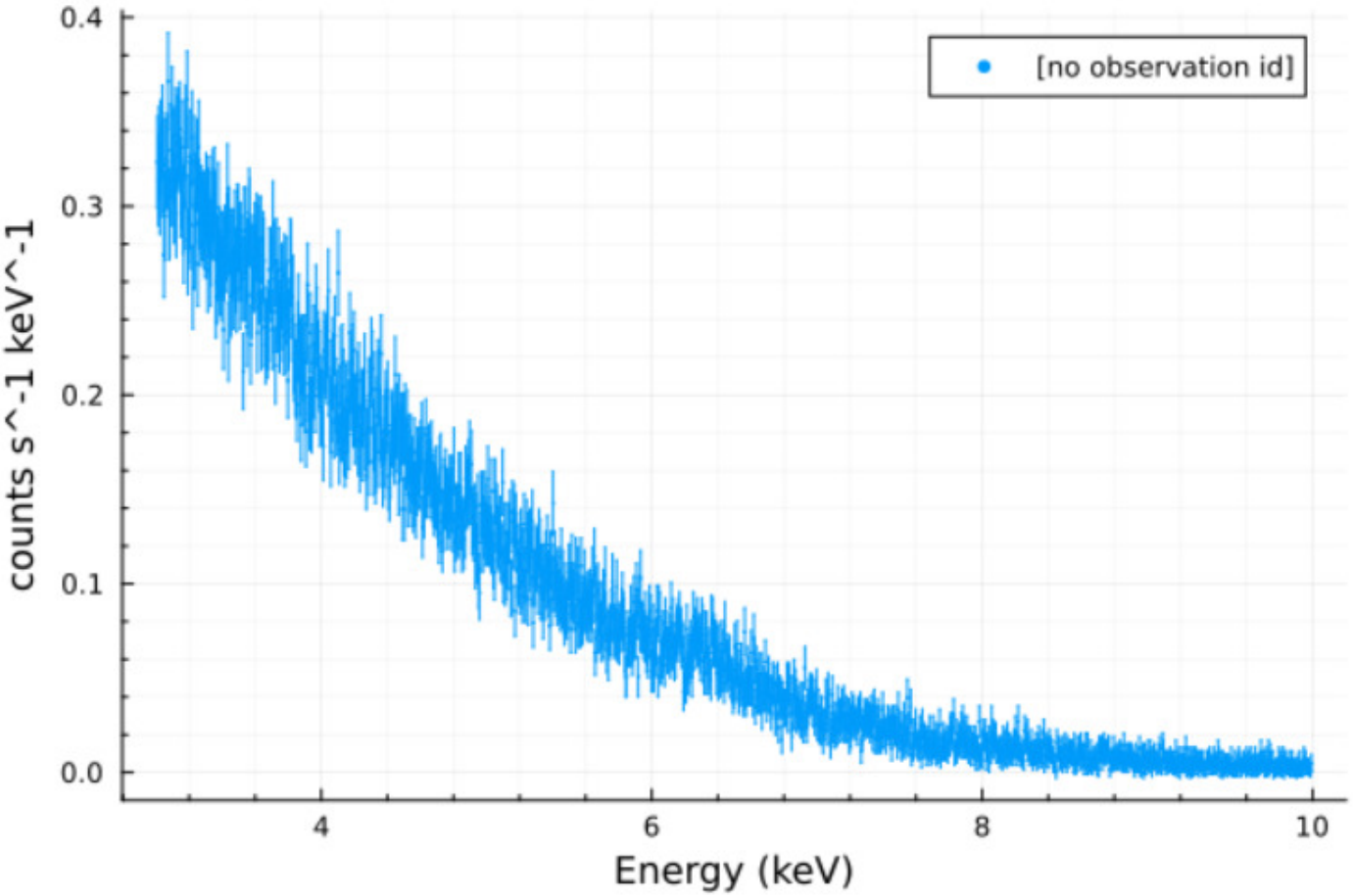


Figure 2.3: The spectrum extracted from the data.

3 Week 3

3.1 2026/10/05

Given that the Julia script was written by somebody else, it was difficult to debug. It was therefore decided that it would be more efficient to rewrite the script in `bash` The script goes to the point of plotting the lightcurves for all three instruments.

The script was added to the shared GitHub repository to download onto solva for testing. A useful command to remember to download a single file from GitHub is:

```
wget --no-check-certificate --content-disposition
 ↪ https://raw.githubusercontent.com/phajy/project-iron-lines/main/code/XMM-Pipeline.sh
```

3.1.1 Comparing the bash script to Julia

The script was tested successfully and the outputs compared to the Julia script. This was tested by looking at the event file produced from the mos1 instrument using `ftlist`.

3.1.1.1 Output from the bash script

```
        Name                Type       Dimensions
        ----                ----       ----------
HDU 1   Primary Array       Null Array
HDU 2   EVENTS              BinTable    12 cols x 70972 rows

  Col  Name              Format[Units](Range)      Comment
    1 TIME               D [s]                 time of the center of the frame
    2 RAWX               I [PIXELS] (-4:605)   X in node coordinates
    3 RAWY               I [PIXELS] (1:602)    Y in node coordinates
    4 DETX               I [pixel] (-19830:19899) X in linearised camera coordinat
    5 DETY               I [pixel] (-20296:19829) Y in linearised camera coordinat
    6 X                  J [pixel] (1.00000000000000E+00:5.18400000000000E+04) X in projected sky co
    7 Y                  J [pixel] (1.00000000000000E+00:5.18400000000000E+04) Y in projected sky co
    8 PHA                I [CHAN] (0:32767)    total charge of the event (uncor
    9 PI                 I [CHAN] (0:15000)    measured energy in eV (gain corr
   10 FLAG               J                     quality flag
   11 PATTERN            B (0:31)              pattern or grade defining the ev
   12 CCDNR              B (1:17)              1st digit=node (0/1), 2nd digit=CCD (1/7)
```

3.1.1.2 Output from the Julia Script

```
        Name               Type       Dimensions
        ----               ----       ----------
HDU 1   Primary Array      Null Array
HDU 2   EVENTS             BinTable     7 cols x 164117 rows

  Col  Name             Format[Units](Range)      Comment
    1 TIME              D [s]                 time of the center of the frame
    2 RAWX              I [PIXELS] (-4:605)   X in node coordinates
    3 PHA               I [CHAN] (0:4095)     total charge of the event (uncor
    4 PI                I [CHAN] (0:15000)    measured energy in eV (gain corr
    5 FLAG              J                     quality flag
    6 PATTERN           B (0:31)              pattern or grade defining the ev
    7 CCDNR             B (1:17)              1st digit=node (0/1), 2nd digit=CCD (1/7)
```

3.1.1.3 Conclusions

It is clear that the Julia script does not compute a step, or does not do so correctly and thus in further analysis the bash script will be used.

3.2 2026/10/07

3.2.1 Meeting with Supervisor

- Suggestions for improving spectrum
  - Plot with more counts per bin
    - Can rebin with `grppha`
  - Plot with a log-log scales
  - Fit with power law and plot ratio of model to data
    - Fabian figure 1
- PN data may have better energy resolution
- Look at more papers on MCG-6-30-15
  - See what authors have done recently
  - Consider the calibration to see if we need to adjust
- Pileup is not important for PN
  - Small-window mode, 70% live time
- MOS may have nicer signal to noise
- CTI - charge transfer inefficiency
  - Some charge gets left behind causing readout streaks
- PSF - shape that a point source takes on the detector
  - XMM relatively blurry
  - Pileup depends on multiple photons hitting the same pixel in a given frame
  - Can exclude the centre of the psf to check for pileup and see if that improves

3.3 2026/10/09

3.3.1 Reprocessing the 0693781301 Dataset Using the PN Instrument

Given the recommendation to use data from the PN instrument, the data was reprocessed. First the bash script was ran with the command

```
/data/cluster4/BradleyAndJacob/Scripts/XMM-Pipeline.sh
```

The resulting lightcurve is shown below

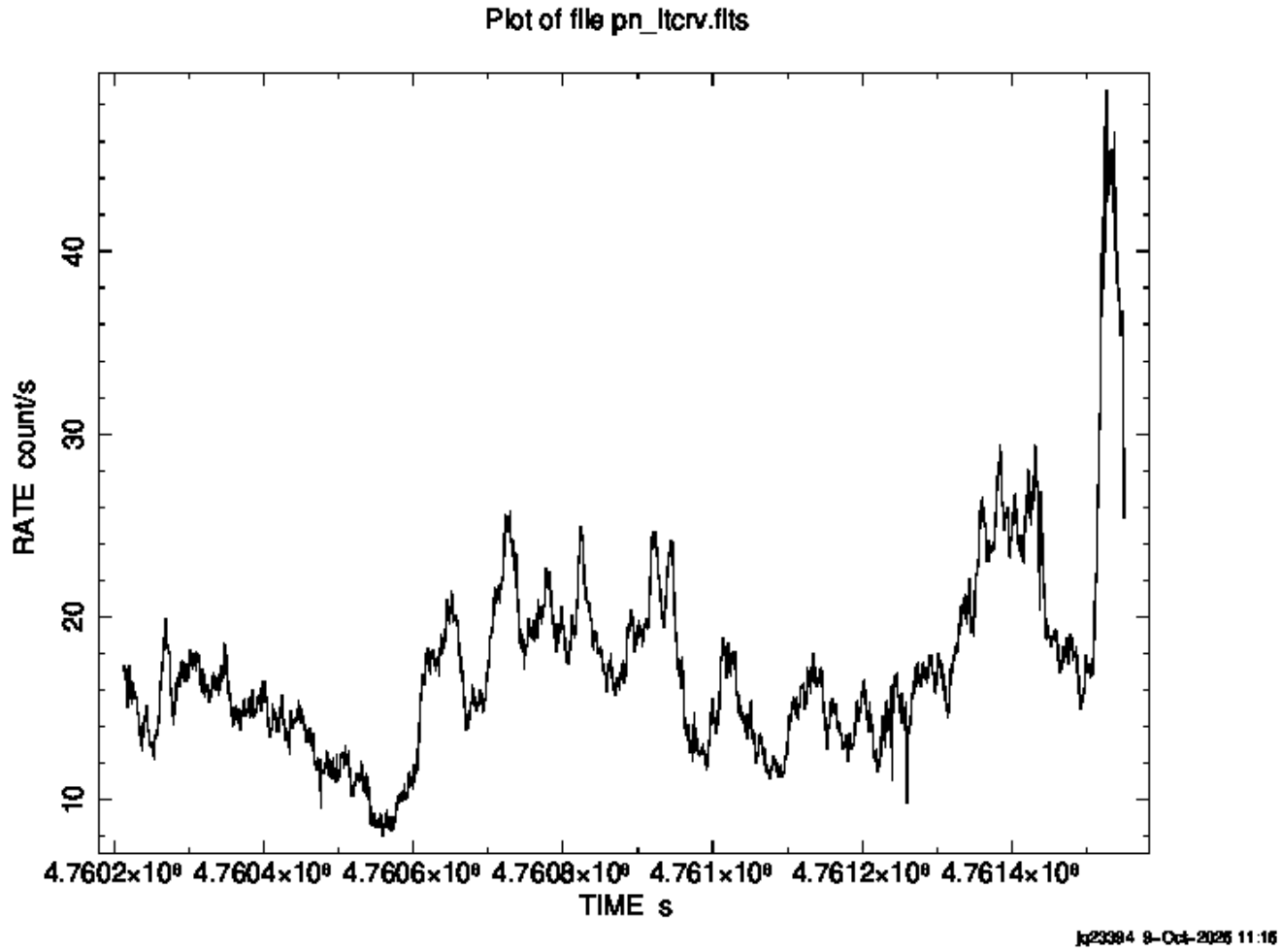


Figure 3.1: The pn lightcurve of the observation. A large flare is again observed at the end of the observation, which was filtered out from the data analysis.

There is once again a flare towards the end so the GTI was set such that RATE is always less than 30

```
# Setting Environment Variables
export SAS_ODFPATH="$(pwd)"
export SAS_CCFPATH="/opt/local/XMM/ccf/"
export SAS_CCF="$SAS_ODFPATH/ccf.cif"
export SAS_ODF="$SAS_ODFPATH/$(ls *SUM.SAS)"
```

```
# Creating the GTI (Good Time Interval) file for time filtering
tabgtigen table=pn_ltcrv.fits gtiset=gtiset.fits expression='RATE<=30'

# Filtering the events file using the GTI
evselect table=pn_filt.fits filtertype=expression filteredset=pn_filt_time.fits
 ↪  expression='GTI(gtiset.fits,TIME)'

# Creating an attitude file
atthkgen atthkset=attitude.fits

# Creating an image for soft X-ray detections
evselect table=pn_filt_time.fits withimageset=yes imageset=pn-s.fits imagebinning=binSize
 ↪  xcolumn=X ximagebinsize=82 ycolumn=Y yimagebinsize=82 filtertype=expression
 ↪  expression='(FLAG == 0)&&(PI in [300:2000])'

# Creating an image for hard X-ray detections
evselect table=pn_filt_time.fits withimageset=yes imageset=pn-h.fits imagebinning=binSize
 ↪  xcolumn=X ximagebinsize=82 ycolumn=Y yimagebinsize=82 filtertype=expression
 ↪  expression='(FLAG == 0)&&(PI in [2000:10000])'

# Creating an image for all X-ray detections
evselect table=pn_filt_time.fits withimageset=yes imageset=pn-all.fits imagebinning=binSize
 ↪  xcolumn=X ximagebinsize=82 ycolumn=Y yimagebinsize=82 filtertype=expression
 ↪  expression='(FLAG == 0)&&(PI in [300:10000])'
```

The regions selected for source and background extraction were centred on (25628, 23912). The source region was a circle with radius 1000 pixels, and the background was an annulus with inner/outer radii 2000/4000 pixels, respectively.

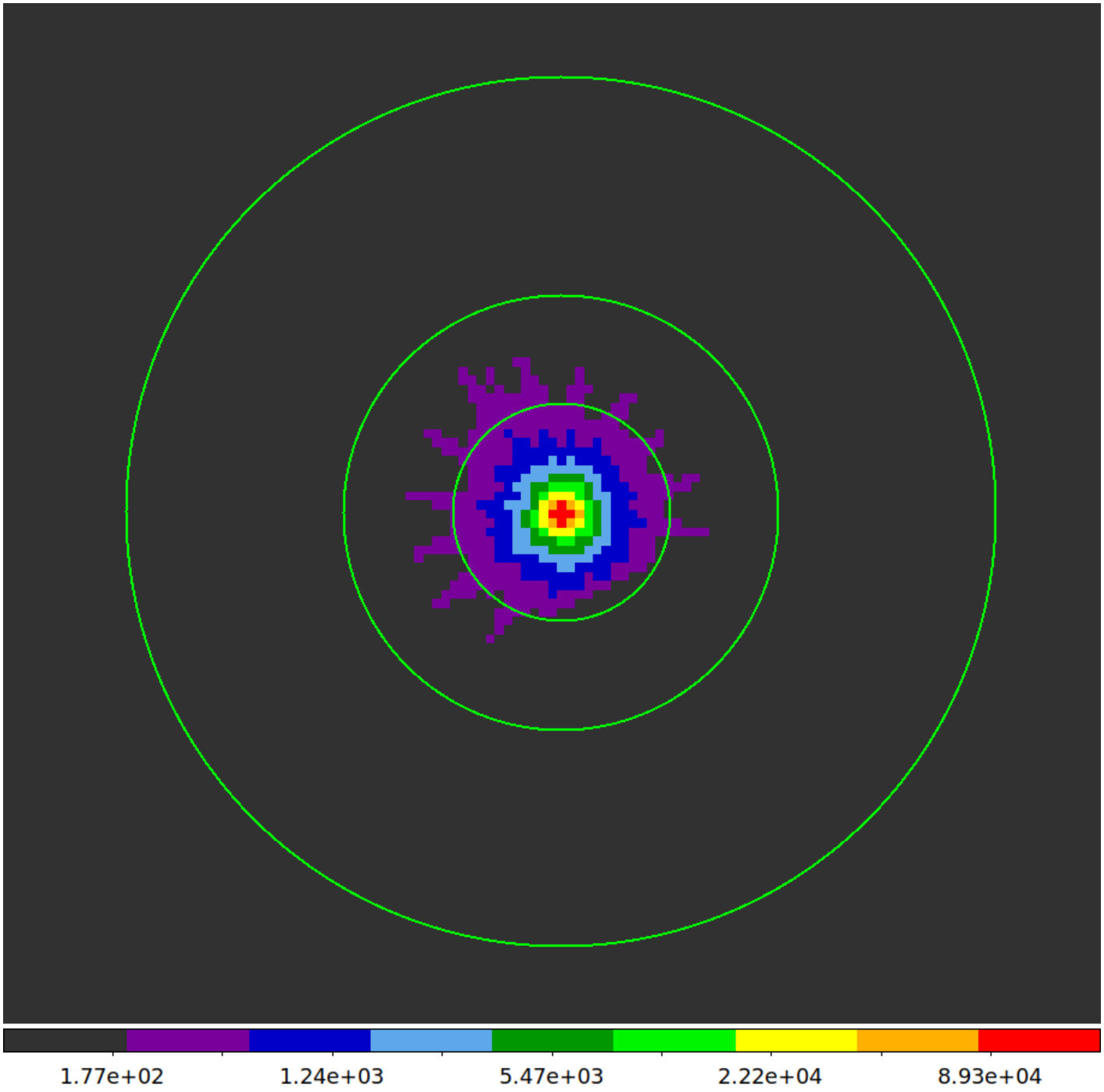


Figure 3.2: The regions selected for source and background extraction. The scale at the bottom denotes how many events were detected in each pixel.

```
eexpmap imageset=pn-h.fits attitudeset=attitude.fits eventset=pn_filt_time.fits
 ↪  expimageset=pn_expmap_2-10keV.fits pimin=2000 pimax=10000

# Extract the Source Spectrum
evselect table='pn_filt_time.fits' energycolumn='PI' filteredset='pn_filtered.fits'
 ↪  filtertype='expression' expression='((X,Y) in CIRCLE(25628,23912,1000))'
 ↪  spectrumset='pn_pi.fits' spectralbinsize=5 withspecranges=yes specchannelmin=0
 ↪  specchannelmax=20479
```

```
# Extract the Background Spectrum
evselect table=pn_filt_time.fits energycolumn='PI' filteredset='bkg_filtered.fits'
 ↪  filtertype='expression' spectrumset='bkg_pi.fits' spectralbinsize=5 expression='((X,Y)
 ↪  in CIRCLE(25628,23912,4000))&&!((X,Y) in CIRCLE(25628,23912,2000))' withspecranges=yes
 ↪  specchannelmin=0 specchannelmax=20479

# Check for Pileup
epatplot set=pn_filtered.fits plotfile=pn_epat.ps useplotfile=yes withbackgroundset=yes
 ↪  backgroundset=bkg_filtered.fits

# Find the source and background region areas for the ancillary file
backscale spectrumset=pn_pi.fits badpixlocation=pn_filt_time.fits
backscale spectrumset=bkg_pi.fits badpixlocation=pn_filt_time.fits

# Generate the Photon Redistribution Matrix
rmfgen rmfset=pn_rmf.fits spectrumset=pn_pi.fits

# Generate the Ancillary File
arfgen arfset=pn_arf.fits spectrumset=pn_pi.fits withrmfset=yes rmfset=pn_rmf.fits
 ↪  withbadpixcorr=yes badpixlocation=pn_filt_time.fits
```

The files were then moved to a products folder with

```
# Making a folder for the products
mkdir ../PRODUCTS

# Storing the observation ID as a variable
export OBSID="0693781301"

# Moving and renaming the product files
cp pn_pi.fits ../PRODUCTS/$(echo $OBSID)_pn_pi.fits
cp bkg_pi.fits ../PRODUCTS/$(echo $OBSID)_pn_bkg_pi.fits
cp pn_rmf.fits ../PRODUCTS/$(echo $OBSID)_pn_rmf.fits
cp pn_arf.fits ../PRODUCTS/$(echo $OBSID)_pn_arf.fits
```

The products could then be plotted, which was once again done with Julia. Per our supervisor's suggestion, the data was rebinned which was completed using the tool `ftgrouppha` as follows:

```
ftgrouppha 0693781301_pn_pi.fits 0693781301_pn_pi_rebin.fits grouptype=min groupscale=400
ftgrouppha 0693781301_pn_bkg_pi.fits 0693781301_pn_bkg_pi_rebin.fits grouptype=min
 ↪  groupscale=400
```

The resulting spectrum is plotted below

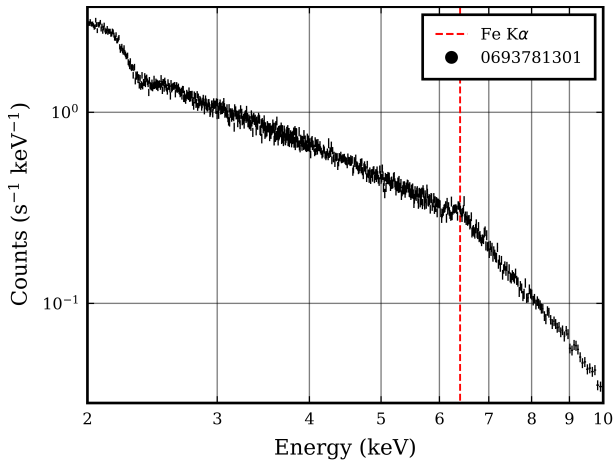


Figure 3.3: The pn spectrum of MCG-6-30-15 from observation ID 0693781301, regrouped to have at least 400 counts per bin. The red line marks 6.4 keV where the Fe Kα emission line is expected. A bump is observed in the expected region.

The next steps are to fit a power law continuum and observe the ratio between the data and the model. This will be done through the XSPEC software, it will be necessary to mask the region around the iron line such that it doesn't interfere with the fit. Similarly, the soft X-ray region below ~2.5 keV will need to be masked.

The response and background files can be linked to the spectrum file, and the rebinning completed using the `ftools` utility `grppha` as follows:

```
grppha
mos1_pi.fits                    # input spectrum filename
mos1_grp30.fits                 # output spectrum filename
chkey BACKFILE bkg_pi.fits      # include background spectrum
chkey RESPFILE mos1_rmf.fits    # include the RMF
chkey ANCRFILE mos1_arf.fits    # include the ARF
group min 300                   # group the data by 300 counts/bin
exit
```

3.4 2026/10/10

3.4.1 A Julia Script for Spectrum Plotting

In order to quickly view the produced spectra, a julia script was produced as shown below. Note that some formatting lines have been excluded for brevity, the full code can be found on the Project GitHub Repository. The code makes use of the `SpectralFitting.jl` library [3].

ViewSpectrum.jl Source Code

```
using SpectralFitting, Plots, LaTeXStrings, ArgParse

# Data loading function
function LoadData(
    spectrum::String;
    dataRange::Tuple{Real, Real}=(2,10)
    )

    # Loading in the spectrum file
    data = OGIPDataset(spectrum)

    # Regrouping, normalising, dropping bad channels and curtailing
    regroup!(data)
    normalize!(data)
    drop_bad_channels!(data)
    mask_energies!(data, dataRange...)
end

function ParseArguments()
    s = ArgParseSettings()

    #Add arguments with their default values
    @add_arg_table s begin
        "--OBS_ID"
            help="The observation ID"
            default="MISSING"
        "--Instrument"
            help="XMM EPIC Instrument name (pn, mos1, mos2)"
            default="pn"
        "--INDIR"
            help="The input directory"
            default=pwd()
        "--OUTDIR"
            help="The output directory"
            default=pwd()
    end

    return parse_args(ARGS, s)
end

args = ParseArguments()

# Loading the data (requires that the spectrum files are grouped)
data = LoadData(
    "$(args["INDIR"])/$(args["OBS_ID"])_$(args["Instrument"])_pi_rebinned.fits"
)

# Setting the observation id
data.user_data.observation_id = args["OBS_ID"]

# Plotting
majorticks = collect(2:10)
vline([6.4]; c=:red, ls=:dash, lw=0.7, label=L"Fe K$\alpha$")
plt = plot!(
    data; ylabel=L"Counts (s$^{-1}$ keV$^{-1}$)",
    xaxis=:log, yaxis=:log, xticks=(majorticks, majorticks)
```

To call the script, the following is used

```
julia PATH_TO_SCRIPT/ViewSpectrum.jl --OBS_ID --Instrument --INDIR --OUTDIR
```

The code currently assumes that the spectrum file follows the convention `[OBS_ID]_[Instrument]_pi.fits` (where `[Instrument]` can be `pn`, `mos1` or `mos2`) and that the spectrum file points to the background and response files (which can be accomplished either with `grppha`, or `specgroup`). The `specgroup` usage follows the syntax

```
specgroup spectrumset=srcspec.ds addfilenames=yes arfset=myarf.ds rmfset=myrmf.ds
 ↪  backgndset=mybgnd.ds
```

3.5 Summary of Data Reduction Commands

Two more bash scripts were made for an intermediate step (`Image-Pipeline.sh`) and the final steps (`Final-Pipeline.sh`) respectively. With these, the data reduction only requires the following commands to be run:

3.5.1 Reprocessing

```
# Run the initial pipeline script
bash /data/cluster4/BradleyAndJacob/Scripts/XMM-Pipeline.sh

# Setting Environment Variables
export SAS_ODFPATH="$(pwd)"
export SAS_CCFPATH="/opt/local/XMM/ccf/"
export SAS_CCF="$SAS_ODFPATH/ccf.cif"
export SAS_ODF="$SAS_ODFPATH/$(ls *SUM.SAS)"
export INSTRUMENT="pn"

# Enter the processing directory
cd ../PROC
```

3.5.2 GTI Filtering

After viewing the lightcurve and determining a filtering expression for a GTI, run

```
# Creating the GTI (Good Time Interval) file for time filtering
tabgtigen table="$INSTRUMENT"_ltcrv.fits gtiset=gtiset.fits expression='RATE<=30'

# Run the image generation pipeline
bash /data/cluster4/BradleyAndJacob/Scripts/Image-Pipeline.sh $INSTRUMENT
```

3.5.3 Region Selection

Then, open the image in ds9 and select the source and background regions

```
# View the image and select regions
ds9 $INSTRUMENT-all.fits &

# Set region expressions as variables, replacing X, Y and R
export SOURCE_CENTRE="X,Y"
export SOURCE_REGION=""$SOURCE_CENTRE",R"
export BACKGROUND_REGION_INNER=""$SOURCE_CENTRE",R"
export BACKGROUND_REGION_OUTER=""$SOURCE_CENTRE",R"
```

3.5.4 Spectrum Extraction

Then enter one of the following pairs of commands, for the relevant instrument

```
# ========== For PN ==========
# Extract the Source Spectrum
evselect table="$INSTRUMENT"_filt_time.fits energycolumn="PI"
 ↪  filteredset="$INSTRUMENT"_filtered.fits filtertype="expression" expression="((X,Y) in
 ↪  CIRCLE("$SOURCE_REGION"))&&(FLAG==0)&&(PATTERN<=4)" spectrumset=""$INSTRUMENT"_pi.fits"
 ↪  spectralbinsize=5 withspecranges=yes specchannelmin=0 specchannelmax=20479

# Extract the Background Spectrum
evselect table="$INSTRUMENT"_filt_time.fits energycolumn="PI"
 ↪  filteredset="bkg_filtered.fits" filtertype="expression" spectrumset="bkg_pi.fits"
 ↪  spectralbinsize=5 expression="((X,Y) in CIRCLE("$BACKGROUND_REGION_OUTER"))&&!((X,Y) in
 ↪  CIRCLE("$BACKGROUND_REGION_INNER"))&&(FLAG==0)&&(PATTERN<=4)" withspecranges=yes
 ↪  specchannelmin=0 specchannelmax=20479
# ========== For PN ==========

# ========= For MOS  =========
# Extract the Source Spectrum
evselect table="$INSTRUMENT"_filt_time.fits energycolumn="PI"
 ↪  filteredset=""$INSTRUMENT"_filtered.fits" filtertype="expression" expression="((X,Y) in
 ↪  CIRCLE("$SOURCE_REGION"))" spectrumset=""$INSTRUMENT"_pi.fits" spectralbinsize=5
 ↪  withspecranges=yes specchannelmin=0 specchannelmax=11999

# Extract the Background Spectrum
evselect table="$INSTRUMENT"_filt_time.fits energycolumn="PI"
 ↪  filteredset="bkg_filtered.fits" filtertype="expression" spectrumset="bkg_pi.fits"
 ↪  spectralbinsize=5 expression="((X,Y) in CIRCLE("$BACKGROUND_REGION_OUTER"))&&!((X,Y) in
 ↪  CIRCLE("$BACKGROUND_REGION_INNER"))" withspecranges=yes specchannelmin=0
 ↪  specchannelmax=11999
# ========= For MOS  =========
```

3.5.5 Pileup Check

Check the spectrum for pileup

```
# Check for Pileup
epatplot set="$INSTRUMENT"_filtered.fits plotfile="$INSTRUMENT"_epat.ps useplotfile=yes
 ↪  withbackgroundset=yes backgroundset=bkg_filtered.fits
```

3.5.6 Response Files and Cleanup

If there is no pileup, run the final script, otherwise reselect the source region to exclude the centre and reduce pileup

```
# If no pileup, run the final pipeline
bash /data/cluster4/BradleyAndJacob/Scripts/Final-Pipeline.sh [OBS_ID] $INSTRUMENT
```

3.5.7 Test Data

This pipeline was tested on the MCG-6-30-15 dataset with observation ID 0111570101. The GTI was set with the expression "TIME>=79698000", abd the regions were centred at (26880,27661) with radii 1000,2000,4000. The resulting spectrum from the pn instrument (after grouping the bins in groups of 4, cutting the data to 1024 channels) is shown below.

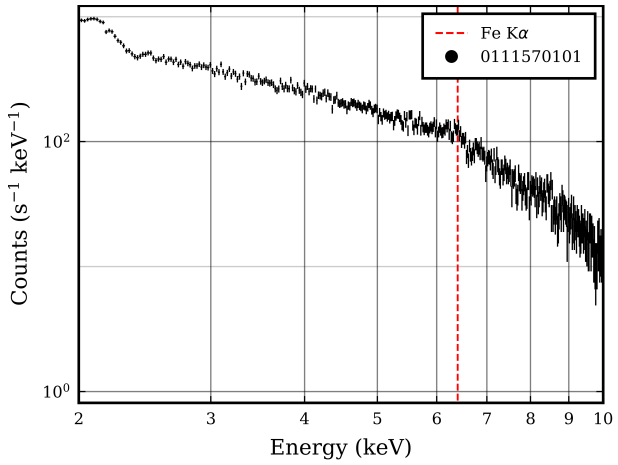


Figure 3.4: The spectrum from obsid 0111570101. The data was once again plotted with ViewSpectrum.jl

This showed a successful spectrum extraction, and yet another promising dataset. The next steps are to thus begin fitting to data in order to reproduce a plot similar to Figure 1 in Fabian et al. (2002) [2].

4 Week 4

4.1 Initial Fitting Attempt Using XSPEC

In order to inspect the data, it will be useful to fit a power law to the continuum excluding the region where we expect the iron line. Taking the ratio between this fit and the data will thus reveal instructive data on the presence of the Fe-Kα line.

The data was rebinned by grouping the channels into sets of 4 via

```
grppha 0693781301_pn_pi.fits 0693781301_pn_pi_rebinned.fits "group 0 4095 4 & exit"
```

The fitting was then completed in XSPEC, fitting a powerlaw. Once the XSPEC session was started (via the `xspec` command), the following sequence of commands was used.

```
# Set plotting parameters and load in the data
cpd 0693781301.ps/CPS
setplot energy
data 0693781301_pn_pi_rebinned.fits

# Setting fitting regions
ignore 0.0-3.0
ignore 15.0-**

# Normalising and ignoring bad channels
renorm
ignore bad

# Plot the data
plot ldata

# Ignore the iron line
ignore 5.0-7.5

# Fit a power law
model powerlaw
fit

# Retrieve the iron line data and plot
notice 5.0-7.5
plot ldata ratio
```

The defaults were used as initial guesses for the fitting parameters (the power law index and the normalisation), and the data was curtailed to be within 3 and 15 keV to capture the best power law region. The resulting fit, and the ratio between the fit and data, are shown below.

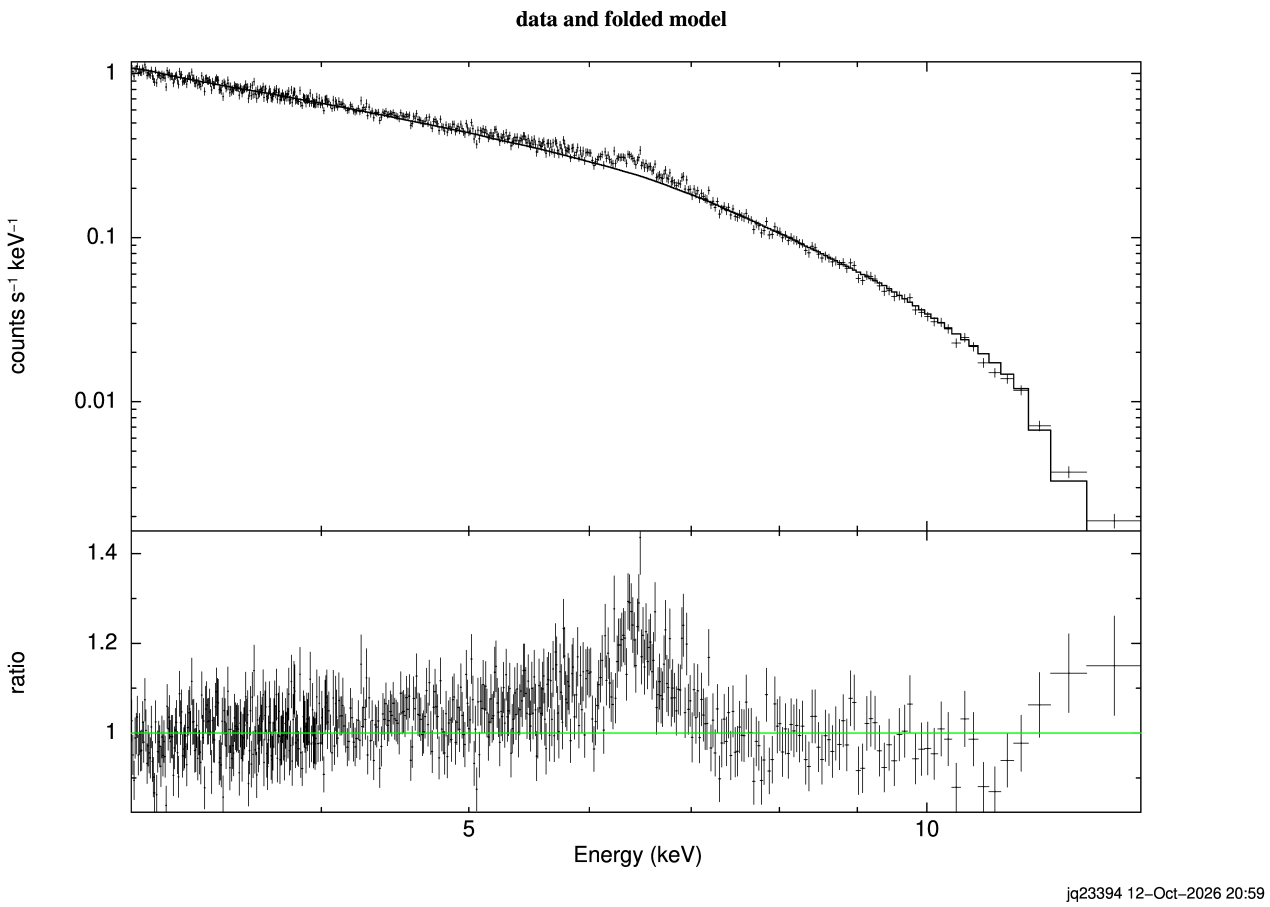


Figure 4.1: The first fit produced using XSPEC. A clear iron line is present at near exactly 6.4 keV.

Further binning may improve this view of the iron line further, which is the next steps.

References

[1] K. Nandra *et al.*, Monthly Notices of the Royal Astronomical Society **382**, 194 (2007).

[2] A. C. Fabian *et al.*, (2002).

[3] F. Baker and A. Young, (2025).